Does the enforcement of legal play through logit masking and environment constraints accelerate agent convergence?

by

Francois Qian
URN: 6702759/7

A dissertation submitted in partial fulfilment of the
requirements for the award of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

May 2026

Department of Computer Science
University of Surrey
Guildford GU2 7XH

Supervised by: Nishanth Sastry

I declare that this dissertation is my own work and that the work of others is acknowledged and indicated by explicit references.

Francois Qian
May 2026

© Copyright Francois Qian, May 2026

Abstract

1. The convergence of self-play in a pseudo-legal environment toward strict legal play.
2. The impact of logit masking on learning dynamics, specifically regarding the "grokking" of physical board mechanics.

In the development of neural agents for combinatorial games, a fundamental tension exists between explicit policy pruning and emergent rule internalization. This thesis investigates two primary paradigms: **Logit Masking** and **Punishment Propagation**.

- **Hypothesis:** While logit masking provides superior convergence speed, unmasked training (Punishment Propagation) forces the network to "grok" the underlying physical mechanics of the environment, potentially yielding more robust latent representations.

Acknowledgements

I would like to thank my supervisor, Nishanth Sastry, for overseeing the project.

Contents

1 Introduction **14**
1.1 Project Background 14
1.2 Project Description 14
1.3 Aims and Objectives 15
1.4 Thesis Structure 15

2 Literature Review **16**
2.1 Chess as a MDP 16
2.2 Chess played by Machines 16
2.2.1 AlphaZero, MuZero 16
2.2.2 Transformers in Chess 17
2.3 Invalid Action Masking 17
2.4 Learning Dynamics 18
2.5 Summary and Research Gap 18

3 System Design and Methodology **19**
3.1 System Architecture Overview 19
3.2 Environment Formulation and State Representation 19
3.2.1 Dual Ruleset MDPs 19
3.2.2 State Canonicalization and Tensor Formulation 19
3.2.3 Network Topology 20
3.2.4 Output Heads 20
3.2.5 Bipartite Monte Carlo Tree Search (MCTS) 20
3.2.6 Selection and PUCT Formulation 20
3.2.7 Exploration Mechanisms 21
3.2.8 Simulation Integrity and Masking 21
3.3 Reinforcement Learning Pipeline 21
3.3.1 Self-Play and Replay Buffer 21
3.4 Optimisation and Loss Function 21
3.5 Experimental Design and Evaluation Metrics 21
3.5.1 The Configuration Matrix 21
3.5.2 Quantitative Metrics 22

4 Implementation Details **23**
4.1 The Chess Client 23

4.1.1 Primitives ... 23
4.1.2 Bitboard ... 23
4.1.3 ChessBoard ... 23
4.1.4 ChessPosition ... 24
4.1.5 ChessGame ... 24
4.2 The Model ... 24
4.2.1 Transformer ... 24
4.2.2 Data ... 24
4.3 The Monte Carlo Tree Search ... 25
4.3.1 Nodes ... 25
4.3.2 Edges ... 25
4.3.3 Arenas ... 25
4.3.4 Engine ... 25
4.4 Main() ... 25

5 Results and Evaluation ... 26
5.1 Experimental Setup ... 26
5.2 Ruleset Convergence Analysis ... 26
5.3 Masking and Grokking Analysis ... 26

6 Conclusion ... 27
6.1 Project Evaluation ... 27
6.2 Limitations and Future Work ... 27

7 Ethics ... 28
7.1 Using Typst ... 28
7.1.1 Adding figures ... 28
7.1.2 Adding tables ... 28
7.1.3 Adding equations ... 28

Bibliography ... 30

List of Figures

Figure 1 An example figure, with two parts 29

List of Tables

Table 1 An example table, showing speed in operations per cycle per multiprocessor **28**

Glossary

$Q(s, a)$	The total reward accumulated from state
$C(s)$	CPUCT
S_i	Upper confidence bound
$\overline{x_i}$	The Mean Value of the current node
c	The Exploitation parameter
N	The current node's Visit Count
n_i	The Visit Count of the child

Abbreviations

MCTS	Monte Carlo Tree Search
UCB1	Upper Confidence Bound 1
PUCT	Predictor + Upper Confidence Bound applied to Trees
CPUCT	Exploration Predictor Constant

Chapter 1

Introduction

1.1 Project Background

Modern machine learning often relies on scaling data and compute over opaque architectures. However, these models are often decoupled from environment transition dynamics, aided by **Logit Masking** - a heuristic that artificially zeroes out the probability of illegal actions prior to softmax activation. While computationally efficient, this deprives the network of feedback for invalid predictions, bypassing the models need to internally represent the mechanics of play. Furthermore, standard RL agents are constrained against making suboptimal or invalid state transitions through environmental constraints. This prevents observation of how internal action space pruning to match a more constrained environment before developing strategy influences learning. Chess provides an optimal substrate for this investigation: it presents highly ambiguous intermediate states bound by objective ground truths and a mathematically perfect, yet computationally intractable, solution space.

1.2 Project Description

Existing literature rarely assesses the impact of these environmental guardrails on the learning dynamics of transformer-based agents. This project investgates two orthogonal axes of environmental constraint: Action Space Constraints and Logit Masking.

To facilitate explicit measurement of model interpretability, a custom two-pass autoregressive transformer encoder and a bipartite Monte Carlo Tree Search (MCTS) are introduced. This architecture decouples piece selection from destination selection, providing a granular window into the model's spatial reasoning.

The experimental configurations are defined across two axes:

1. **Axis 1: Rule Set Convergence (Action Space Constraints)**
   - **Control:** Training on a strict legal ruleset.
   - **Test:** Training on a pseudo-legal ruleset where the terminal state is the simpler, denser king capture $\left(K_{\text{cap}}\right)$ over the highly sparse, complex $(A_l = \emptyset)$. This variant of Chess fundamentally alters the MDP reward function to $R(s, a)$ yielding a potentially smoother reward landscape.

- **Hypothesis:** The network first bootstraps learning the rules and goals of the environment. Ideal play in the pseudo-legal space A_P may converge to standard Chess where concepts such as pins and checkmates emerge as survival heuristics to prevent or secure king capture. Conversely, the removal of environmental guardrails may degrade MCTS value estimation, preventing policy bootstrapping.

2. **Axis 2: Logit Masking**
   - **Control:** Masked logits (illegal moves are mathematically eliminated prior to selection).
   - **Test:** Unmasked logits (illegal moves are permitted by the network but penalized by the environment/loss function).
   - **Hypothesis:** Unmasked training will incur an initial sample-efficiency penalty by introducing the dual-optimization problem of learning game mechanics alongside strategy, but may lead to more robust internal representations reflecting potentially stronger play long term.

1.3 Aims and Objectives

The primary aim of this project is to train multiple transformer-based agents via self-play reinforcement learning across varying rule sets and action spaces, quantitatively evaluating their performance, learning trajectories, and emergent behaviors.

Specific objectives include:

- Implement a highly optimized, dual-ruleset (legal and pseudo-legal) Chess library to facilitate high-throughput RL self-play.
- Architect and implement a two-pass autoregressive transformer and bipartite MCTS to explicitly model piece-then-square action selection.
- Train four distinct model configurations via self-play: (Legal, Masked), (Legal, Unmasked), (Pseudo-Legal, Masked), and (Pseudo-Legal, Unmasked).
- Quantitatively evaluate the convergence rates, illegal move proposal frequencies, and the emergence of rule-abiding play across the unconstrained configurations.
- Correlate the emergence of legal play with the attention distributions of the 2 pass encoder to quantify spatial reasoning.

1.4 Thesis Structure

The remainder of this dissertation is structured as follows:

- **Chapter 2** reviews the literature surrounding reinforcement learning, transformer architectures, and action masking.
- **Chapter 3** details the system design, including the bipartite MCTS and two-pass encoder architecture.
- **Chapter 4** outlines the implementation details of the Rust engine and training pipeline.
- **Chapter 5** presents the results and evaluation of the four experimental configurations.
- **Chapter 6** concludes the project and discusses avenues for future work.

Chapter 2

Literature Review

2.1 Chess as a MDP

Chess is a deterministic, zero-sum, perfect-information game. It is formally modeled as a Markov Decision Process (MDP) defined by a tuple (S, A, T, R), where S is the state space ($|S| \approx 10^{\{40\}}$), A is the action space, $T : S \times A \to S$ is the deterministic transition function, and $R : S \times A \to \{-1, 0, 1\}$ is the reward function (Shannon, 1950).

Standard RL implementations enforce a legal action space $A_{L(s)} \subset A$ where A_L contains only moves that comply with the rules of chess (e.g., preventing self-check) (World Chess Federation (FIDE), 2023). This project introduces a relaxed pseudo-legal action space $A_{P(s)}$ where $A_{L(s)} \subset A_{P(s)} \subset A$, permitting all geometrically valid piece movements and shifting the terminal condition from the abstract checkmate to the explicit king capture $\left(K_{\text{cap}}\right)$.

Typical agents produce a single policy distribution representing every possible piece movement on all 64 squares ($|A| = 4672$). A standard chess position has approximately 30 legal moves ($|A_{L(s)}| \approx 30$) (Shannon, 1950), vastly smaller than the unrestricted action space. Factoring the joint probability $P(a \mid s)$ into $P(\text{from} \mid s) \times P(\text{to} \mid s, \text{from})$ reduces output layer parameters significantly (e.g., from d_model $\times$ 4672 to d_model $\times$ 64).

For any state s, an optimal policy π^* trained in A_P must implicitly learn to prune the action space, such that:

$$\pi^*(a \mid s) \to 0 \quad \text{for all } a \in \left(A_{P(s)} \setminus A_{L(s)}\right)$$

In this paradigm, concepts such as pins and checkmates are not hardcoded environmental constraints, but emergent survival heuristics within the value landscape.

2.2 Chess played by Machines

2.2.1 AlphaZero, MuZero

The paradigm of self-play RL in chess was defined by AlphaZero (Silver *et al.*, 2017), which combined Monte Carlo Tree Search (MCTS) with deep convolutional neural networks (CNNs) for policy and value evaluation. However, AlphaZero relies entirely on a hardcoded

environment to generate $A_{L(s)}$, bypassing the need for the network to internalize the rules of the game.

Subsequent architectures, such as MuZero (Schrittwieser *et al.*, 2020), removed the need for a known transition dynamics model by learning a latent environment simulator. While MuZero demonstrates the ability to adhere to environment constraints implicitly within its hidden state transitions, its policy head still simulates within the legal action space during MCTS rollouts. The literature lacks insight into how the removal of environmental guardrails or action masking impacts the sample efficiency and representational robustness of the underlying network.

2.2.2 Transformers in Chess

Recent advancements have demonstrated the efficacy of Transformer architectures in chess, shifting away from the spatial inductive biases of CNNs. (Monroe and Chalmers, 2024) demonstrated that Transformers can achieve Grandmaster-level performance without explicit search, relying entirely on attention mechanisms to evaluate board states. Furthermore, (Jenner *et al.*, 2024) provided evidence of learned look-ahead within chess-playing neural networks, showing that attention heads naturally encode future board states.

Standard chess engines flatten the action space into a single discrete distribution. Decoupling piece selection $P(\text{from} \mid s)$ and destination selection $P(\text{to} \mid s, \text{from})$ forces attention mechanisms to explicitly map geometric constraints. This factored approach provides an observable window into distinct failure modes: attention failure (identifying movable pieces) and kinematic failure (how pieces move).

2.3 Invalid Action Masking

In policy gradient algorithms, it is standard practice to prevent the selection of illegal actions via Invalid Action Masking (Logit Masking). Masking applies a transformation to the policy logits z prior to the softmax activation:

$$P(a_i \mid s) = \frac{\exp(z_i + M_i)}{\sum_j \exp\big(z_j + M_j\big)}$$

where $M_i = -\infty$ if $a_i \notin A_{L(s)}$, and 0 otherwise.

While often employed to accelerate convergence and simplify design, (Huang and Ontañón, 2020) demonstrated that invalid action masking significantly improves sample efficiency and asymptotic performance in environments with large, state-dependent action spaces by collapsing the exploration space over the manifold space. (Kanervisto, Scheller and Hautamäki, 2020) similarly emphasized the importance of action space transformations within policy gradient algorithms to create learnable environments, especially for continuous action spaces. However, masking prevents punishment from invalid actions from propagating back through the network, effectively disabling feedback for invalid decisions. This is particularly critical for MCTS algorithms, where simulations into invalid state transitions degrade search efficiency and poison the policy head.

2.4 Learning Dynamics

The requirement for an unmasked network to learn the rules of chess from scratch introduces complex learning dynamics. (Power *et al.*, 2022) identified the phenomenon of "grokking," where neural networks trained on small algorithmic datasets exhibit delayed generalization long after overfitting the training data. This phenomenon relies on weight decay pressure, signifying the importance of weight regularization in network robustness. Chess is similarly an algorithmic dataset embedded within a strategic MDP, showcasing the potential for analogous dynamics.

(Bengio *et al.*, 2009) formalized curriculum learning, finding that machine learning models converge more effectively if training data is organized in a sequence of increasing complexity. The authors demonstrate this strategy effectively functions as a regularization method, guiding the model to superior local minima often inaccessible through data shuffling. Theoretically, a pseudo-legal environment serves as an implicit curriculum, where an agent first receives rewards through piece kinematics and king capture before developing legal survival heuristics.

(Sharma *et al.*, 2017) introduced a framework that exploits the compositional nature of discrete action spaces by factoring them into independent basis-like components. This enables cross-action learning, allowing the agent to generalize actions that share common factors.

2.5 Summary and Research Gap

While the efficacy of action masking is well-documented in general RL (Huang and Ontañón, 2020), and Transformers have proven capable of modeling chess (Jenner *et al.*, 2024), the intersection of these domains remains unexplored. Existing literature does not isolate the impact of environmental guardrails on the representational quality of Transformer-based MCTS agents.

In the context of unmasked, pseudo-legal chess training, the network faces a dual-optimization problem: it must learn the physical constraints of the board (the rules) and the strategic evaluation of states (the game). It is hypothesized that the network will exhibit a grokking-like phase transition: an initial period of high illegal-move proposal (random walk), followed by a sharp convergence toward $A_{L(s)}$ as the physical constraints are internalized, preceding the development of high-level strategy.

This project addresses this gap by training four distinct configurations across two orthogonal axes: Action Space Constraints (Legal vs. Pseudo-Legal) and Logit Masking (Masked vs. Unmasked). By utilizing a factored two-pass autoregressive encoder, this research provides a novel, quantitative analysis of how rule internalization and action space pruning manifest within the attention layers of a chess-playing agent.

Chapter 3

System Design and Methodology

3.1 System Architecture Overview

The system is designed as a closed-loop reinforcement learning pipeline, decoupling the environment transition dynamics from the neural network's internal representations. The architecture consists of three primary components: a custom deterministic environment simulator supporting dual rulesets, a two-pass autoregressive Transformer encoder, and a bipartite Monte Carlo Tree Search (MCTS) algorithm. The system operates via asynchronous self-play, generating trajectories that are aggregated into a global replay buffer to optimize the network via gradient descent.

3.2 Environment Formulation and State Representation

3.2.1 Dual Ruleset MDPs

1. **Legal Environment ($\boldsymbol{E_l}$)**: Adheres to standard FIDE rules. The action space ($A_{l(s)}$) is strictly contained to moves that do not leave the king in check. The terminal reward $R(s, a)$ is evaluated upon Checkmate, Stalemate or standard draw conditions.
2. **Pseudo-Legal Environment ($\boldsymbol{E_p}$)**: Relaxes the action space ($A_{p(s)}$), permitting all geometrically valid piece movements regardless of king safety. The terminal conditions is shifted to the explicit king capture (K_{cap}). FIDE draw conditions are maintained to prevent infinite rollouts and support correct transition into legal play (e.g., stalemate by no legal moves which would otherwise lead to a win in pseudo-legal play).

3.2.2 State Canonicalization and Tensor Formulation

To enforce symmetric learning and halve the state space, all board states are canonicalized to the perspective of the side-to-move. If it is Black's turn, the board and castling rights are geometrically flipped. The state s is mapped to the spatial tensor representation $X \in \mathbb{R}^{64\times 14}$ and a scalar meta-tensor $M \in \mathbb{R}^5$.

- **Spatial Tensor**: Comprises 12 piece bitboards (6 piece types $\times$ 2 colors), 1 en-passant target plane, and 1 selected-square plane (utilized in the second pass of the network).
- **Meta Tensor**: Encodes the 4 castling rights (one-hot) and the half-move clock (scaled by $\frac{1}{100}$).

Standard Chess architectures flatten the action space into a single discrete distribution. To explicitly measure spatial reasoning and interpretability, this system factors the joint probability of a move $P(a|s)$ into an autoregressive sequence: $P(a|s) = P(\text{from}|s) \times P(\text{to}|s, \text{from})$.

The network executes 2 forward passes using shared weights:

1. **Pass 1 (Piece Selection)**: The network evaluates the board state with the selected square plane zeroed out, outputting a policy distribution over the 64 squares to select the origin square (*from*). The value head will evaluate the board state.
2. **Pass 2 (Destination Selection)**: The origin square is encoded into the 14th plane of the spatial tensor. The network re-evaluates the state, outputting a policy distribution over the 64 squares to select the destination (*to*). The value head will evaluate the viability of the selected piece.

3.2.3 Network Topology

The model uses a Transformer Encoder architecture without spatial inductive biases (e.g., convolutions). The spatial tensor X is linearly projected to d_{model}, and $2D$ learned positional embeddings are added to retain geometric context. The meta-tensor M is linearly projected and concatenated as a pseudo-[CLS] token, resulting in a sequence length of 65.

The model is scaled symmetrically:

- $n_{\text{heads}} = 8$
- $n_{\text{layers}} = 8$
- $d_{\text{model}} = 8 * 64$
- $d_{\text{ff}} = 8 * 4 * 64$

3.2.4 Output Heads

The final latent representation is routed into two distinct heads:

1. **Policy Head**: A linear projection of the 64 spatial tokens to $\mathbb{R}^{64}$, representing unnormalised logits for square seletion.
2. **Value Head**: A linear projection of the [CLS] token to $\mathbb{R}^3$, passed through softmax acti

3.2.5 Bipartite Monte Carlo Tree Search (MCTS)

To accomodate the two-pass autoregressive policy, a bipartite MCTS is implemented. The search tree alternates between two node types:

- **Selection Nodes (N_{select})**: Represents a board state. Edges represent selection of an origin square.
- **Action Nodes (N_{move})**: Represents a board state plus selected origin square. Edges represent the destination square, transitioning the environment to an N_{select} node.

3.2.6 Selection and PUCT Formulation

During traversals, edges are selected using a modified Predictor + Upper Confidence Bound applied to Trees (PUCT) algorithm. To mitigate draw-seeking behaviour, a contempt factor is integrated into the exploitation term:

$$U(s,a) = (W - L - 0.05D)(\text{Exploitation}) + c_{\text{puct}} P(s,a) \frac{\sqrt{\sum_b N(s,b)}}{1+N(s,a)} (\text{Exploration})$$

where W, D and L are the mean value estimates for the edge, P(s, a), is the prior probability from the policy head, and N is the visit count.

3.2.7 Exploration Mechanisms

To ensure robust exploration of the state space:

1. **Dirichlet Noise**: Applied exclusively to the root node prior probabilities: $P(s, a) = (1 - \varepsilon)P(s, a) + \varepsilon\eta$, where $\eta \sim \mathrm{Dir}(\alpha)$ with $\alpha = 0.3$ and $\varepsilon = 0.25$.
2. **Temperature Scaling**: Move selection at the node is sampled proportionally to $N(s, a)^{\frac{1}{t}}$. The temperature t scales dynamically based on ply count and remaining material, transitioning from exploratory in the opening to deterministic in endgames.

3.2.8 Simulation Integrity and Masking

During MCTS rollouts, invalid state transitions poison the value estimations of the tree. To maintain simulation integrity, action masking is applied during the search phase across all configurations. However, in the "Unmasked" experimental configurations, the network's raw logits are not masked during the calculation of the loss function, forcing the network to internalize the penalty for assigning probability mass to illegal moves.

3.3 Reinforcement Learning Pipeline

3.3.1 Self-Play and Replay Buffer

The system generates data via highly parallelized self-play. Trajectories are stored in a First-In-First-Out (FIFO) replay buffer with a capacity of 524,288 samples. To prevent infinite games, rollouts are forcibly terminated as draws after 400 plies, or if the network's internal value estimation for a draw exceeds 0.95, scaled to 0.75 in the endgame.

3.4 Optimisation and Loss Function

The network is optimised using the AdamW optimiser with weight decay (10^{-4}) and a Noam learning rate scheduler. The loss function is a linear combination of policy loss and value loss.

For a given state s, let π be the MCTS visit count distribution, p be the network's policy prediction, z be the true game outcome, and v be the network's value prediction. The loss function is defined as: $L = \lambda\left(-\sum_a \pi(a|s)\log p(a|s)\right) + (1-\lambda)\left(-\sum_i z_i \log v_i\right)$ where $\lambda = 0.7$ weights the policy gradient against the value estimation. In unmasked configurations, $p(a|s)$ includes probability mass on illegal moves, which are penalised as $\pi(a|s) = 0$ for all invalid actions.

3.5 Experimental Design and Evaluation Metrics

3.5.1 The Configuration Matrix

To isolate the effects of environmental guardrails, four distinct models are trained from scratch using identical hyperparameters (256 batch size, 256 MCTS simulations per move). The configurations form a 2×2 matrix:

1. **Legal + Masked (Control)**: Standard AlphaZero paradigm.
2. **Legal + Unmasked**: Network must learn piece kinematics via loss penalties before strategy.

3. **Pseudo-Legal + Masked**: Network plays the K_{cap} variant, and must learn FIDE rules before strategy.
4. **Pseudo-Legal + Unmasked**: Network must learn piece kinematics and FIDE rules entirely from scratch.

3.5.2 Quantitative Metrics

The learning dynamics of each configuration are evaluated using the following metrics, logged continuously during training:

- **Average Illegal Probability**: The total probability mass assigned by the raw policy head to invalid actions, quantifying the network's internalization of the ruleset.
- **Average Game Length**: Measures in plies, indicating the transition from random walks to strategic, prolonged play.
- **Win/Draw Ratios**: Tracking the emergence of decisive outcomes versus stalemates or repetitions.
- **Value Loss Convergence**: Measures the accuracy of the network's state evaluation over time.

Chapter 4

Implementation Details

4.1 The Chess Client

4.1.1 Primitives

- ChessSquare
- ChessMove
- Color
- CaslingRights
- Zobrist Hashing

4.1.2 Bitboard

The board is represented using 12 primary bitboards (unsigned 64-bit integers), packing the state into 96 bytes. Additional convenience bitboards (occupancy masks) are maintained to accelerate move generation. Bitboards enable high-performance move generation via bitwise intrinsics. In deep learning, these are expanded into 64×14 tensors.

- Consts

4.1.3 ChessBoard

```
struct ChessBoard {
    pieces: [[Bitboard; 6]; 2],
    white_occupancy: Bitboard,
    black_occupancy: Bitboard,
    all_pieces: Bitboard,
}
```

4.1.3.1 Masks

```
const WHITE_SQUARES: Bitboard;
const BLACK_SQUARES: Bitboard;
const KNIGHT_ATTACKS: [Bitboard; 64];
const KING_ATTACKS: [Bitboard; 64];
const PAWN_ATTACKS_WHITE: [Bitboard; 64];
const PAWN_ATTACKS_BLACK: [Bitboard; 64];

// SOUTH, WEST, NORTH, EAST
pub const ROOK_ATTACKS: [[Bitboard; 4]; 64];
```

```
// SW, SE, NE, NW
const BISHOP_ATTACKS: [[Bitboard; 4]; 64];
const BISHOP_ATTACKS_ALL: [Bitboard; 64];
const ROOK_ATTACKS_ALL: [Bitboard; 64];
const BETWEEN: [[Option<Bitboard>; 64]; 64];
```

4.1.4 ChessPosition

```
struct ChessPosition {
    chessboard: ChessBoard,
    side_to_move: Color,
    castling_rights: CastlingRights,
    en_passant: Option<ChessSquare>,
    halfmove_clock: u32,
    zobrist_hash: u64,
    pseudolegal_moves: ArrayVec<ChessMove, 128>,
}
```

4.1.4.1 Move Generation

Ray casting is optimized using constant directional masks and pre-computed "between" masks, allowing for $O(1)$ validation of sliding piece moves.

4.1.5 ChessGame

```
struct ChessGame {
    castling_rights: CastlingRights,
    chessboard: ChessBoard,
    en_passant: Option<ChessSquare>,
    fullmove_counter: u32,
    game_history: alloc::vec::Vec<GameStateEntry>,
    halfmove_clock: u32,
    outcome: Outcome,
    rule_set: RuleSet,
    side_to_move: chess_piece::Color,
    zobrist_hash: u64,
}
```

4.2 The Model

4.2.1 Transformer

4.2.2 Data

3 fold repetition is handled by the environment rather than being encoded as an input. the replay buffer is initialised with size 524288, meaning the model learns from the past approx 500000 moves made. with a batch size of 256, 2048 moves are made before saturating the replay buffer. assuming an average game length of 96, the model learns from approx the last 21 games played at a time, balancing forgetting with data freshness.

4.3 The Monte Carlo Tree Search

4.3.1 Nodes

4.3.1.1 Select

4.3.1.2 Move

4.3.2 Edges

4.3.3 Arenas

4.3.4 Engine

4.4 Main()

Chapter 5

Results and Evaluation

5.1 Experimental Setup

To ensure fairness, all variants were initialized with identical seeds, trained for the same number of epochs, and evaluated at identical MCTS search depths.

5.2 Ruleset Convergence Analysis

5.3 Masking and Grokking Analysis

Chapter 6

Conclusion

6.1 Project Evaluation

6.2 Limitations and Future Work

- **Compute Constraints:** Hardware limitations restricted the total number of training epochs, preventing observation of

deep late-stage grokking.

- **Move Generation:** Future iterations should replace standard bitboard raycasting with Magic Bitboards for optimal

performance.

- **Endgame Tablebases (EGTB):** Injecting perfect knowledge at varying stages of the training pipeline remains a highly viable area for future research to bootstrap the value head and sharpen endgame play.
- **Kinematic bootstrapping:** Boot strap the policy head with uniform distributions of valid moves, before playing for value.

Chapter 7

Ethics

7.1 Using Typst

Operation	Speed
Add, Mul, Mul-Add	8
Reciprocal	2
Divide	0.88
Divide Intrinsic	1.6
Recip. Square Root	2
Square Root	1
Logarithm	2
Exponent	1
Sin, Cos Intrinsics	2
Sin, Cos, Tan	Slow

Table 1: An example table, showing speed in operations per cycle per multiprocessor

7.1.1 Adding figures

Figures are added using the `#figure` function. Typst automatically handles the numbering and placement of your images, as shown in the example that produces Figure 1.1. Common formats like PNG, JPEG, and SVG are supported natively.

7.1.2 Adding tables

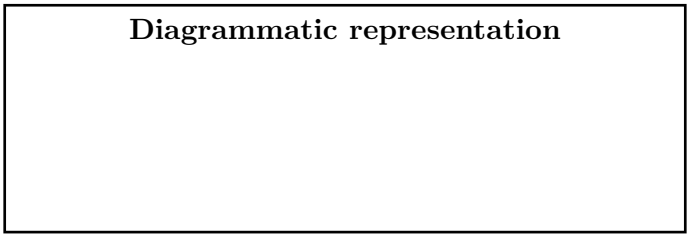
Tables are defined directly in the source code using the `#table` function. This provides a highly flexible way to grid your data. An example of a styled table is given in Table 1.1.

7.1.3 Adding equations

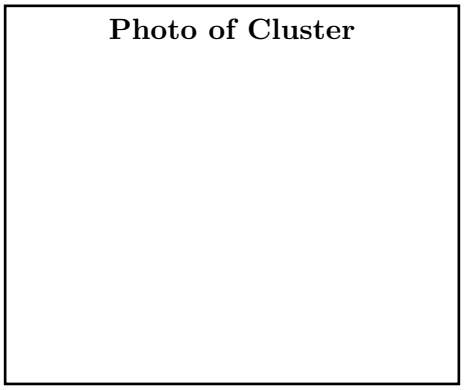
A primary advantage of Typst is its intuitive mathematical notation. Equations can be written within `$ ... $` delimiters. It handles numbered equations easily, as in the recursive formula:

$$\alpha(\iota, x_2) = \sum_{x_1, d_{\iota-1}} \alpha(\iota - 1, x_1) \Pr\left\{\boldsymbol{r}_{n(\iota-1)+x_1, n\iota+x_2}, \boldsymbol{t}_{\iota-1}\right\}$$

Inline math is also supported, for example to specify $1 \leq \iota < N$. Typst's syntax remains readable even for complex multiline expressions: testing testings



(a) Diagrammatic representation



(b) Photo of the Tempest cluster

Figure 1: An example figure, with two parts

$$\Pr\left\{\boldsymbol{r}_{0,n\iota+x_2}, \sigma_{n\iota} = x_2\right\} = \sum_{x_1, d_{\iota-1}} \left[\Pr\left\{\boldsymbol{r}_{0,n(\iota-1)+x_1}, \sigma_{n(\iota-1)} = x_1\right\} \times \Pr\left\{\boldsymbol{r}_{n(\iota-1)+x_1,n\iota+x_2}, \boldsymbol{t}_{\iota-1}\right\}\right]$$

Bibliography

Bengio, Y. *et al.* (2009) "Curriculum learning," in *Proceedings of the 26th Annual International Conference on Machine Learning*. Montreal, Quebec, Canada: Association for Computing Machinery (ICML '09), pp. 41–48. Available at: https://doi.org/10.1145/1553374.1553380.

Huang, S. and Ontañón, S. (2020) "A Closer Look at Invalid Action Masking in Policy Gradient Algorithms," *CoRR* [Preprint]. Available at: https://arxiv.org/abs/2006.14171.

Jenner, E. *et al.* (2024) *Evidence of Learned Look-Ahead in a Chess-Playing Neural Network*. Available at: https://arxiv.org/abs/2406.00877.

Kanervisto, A., Scheller, C. and Hautamäki, V. (2020) "Action Space Shaping in Deep Reinforcement Learning," in *2020 IEEE Conference on Games (CoG)*, pp. 479–486. Available at: https://doi.org/10.1109/CoG47356.2020.9231687.

Monroe, D. and Chalmers, P.A. (2024) *Mastering Chess with a Transformer Model*. Available at: https://arxiv.org/abs/2409.12272.

Power, A. *et al.* (2022) "Grokking: Generalization beyond overfitting on small algorithmic datasets," *arXiv preprint arXiv:2201.02177* [Preprint].

Schrittwieser, J. *et al.* (2020) "Mastering Atari, Go, chess and shogi by planning with a learned model," *Nature*, 588(7839), pp. 604–609. Available at: https://doi.org/10.1038/s41586-020-03051-4.

Shannon, C.E. (1950) "XXII. Programming a computer for playing chess," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314), pp. 256–275. Available at: https://doi.org/10.1080/14786445008521796.

Sharma, S. *et al.* (2017) "Learning to Factor Policies and Action-Value Functions: Factored Action Space Representations for Deep Reinforcement learning," *CoRR* [Preprint]. Available at: http://arxiv.org/abs/1705.07269.

Silver, D. *et al.* (2017) *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. Available at: https://arxiv.org/abs/1712.01815.

World Chess Federation (FIDE) (2023) "FIDE Laws of Chess". Available at: https://handbook.fide.com/chapter/E012023.